CT-208 PERMUTATIONS

Contents

Part 1. Permutation Notations: be careful 1
1. Interpretation of table or lexical notation 1
2. Non-comutativity of the permutation group 2
3. Cicle notation 2
4. Problems and Exercises 3

Part 2. How to apply a permutation *in situ* 4
5. Problem definition 4
6. An algorithm for permuting in situ 5
7. Complexity Analysis: Preliminaries 5
7.1. Canonical cycle notation 5
7.2. Preliminaries 6
7.3. Worst Case 6
8. Complexity Analysis: Average Case of a 7
8.1. The y_{ij} function 8
8.2. Calculating $p(y_{ij} = 1)$ 8
9. Exercises 9

Part 3. Algorithms to generate all permutations 10
10. Inversions (TAOCP Vol. 3 §5.1.1) 10
11. The L Algorithm: Lexicographical ordering 11
11.1. Overview of the L algorithm 11
11.2. Improved L Algorithm 11
12. The P Algorithm (Gray code-like) 13
12.1. The Improved P Algorithm 14
13. The X algorithm: lexicographical ordering with skipping 14
14. Exercises and other algorithms 15
15. Conventions for comparing algorithms to generate all permutations 18

Part 1. Permutation Notations: be careful

1. Interpretation of table or lexical notation

When permutations are represented by tables, they must be interpreted in a consistent way. E.g.:

index	1	2	3
π	3	2	1
$\mathscr{L}$	e	z	d

To compute a permutation of $\mathscr{L}$, we must apply π over it, generating another vector , $\mathscr{L}$'. It can easily be done in O(n), by appling sequentially one of these two equations: $\mathscr{L}'[i] = \mathscr{L}[\pi(i)]$ or $\mathscr{L}'[\pi(i)] = \mathscr{L}[i]$. Check that the end result is the same for both options: $\mathscr{L}' = [d, z, e]$.

But, consider this other example:

index	1	2	3	4
π	4	3	1	2
$\mathscr{L}$	e	z	d	a

If we interpret the table differently when applying the permutation, now we obtain two different lists: $\mathscr{L}' = [a, d, e, z]$ and $\mathscr{L}' = [d, a, z, e]$. In informal language, the first interpretation corresponds to reading the table as "first, comes the fourth element. second, comes the third element; so on.". While the second interpretation would read the table as "the first value goes to the fourth position; the second value goes to the third position; so on." From now on, we will use the second interpretation.

2. Non-comutativity of the permutation group

Composition of permutations form a group[1], but a non-comutative one. Different references may interpret $\alpha \cdot \beta$ differently. E.g.[2], Rotman's book considers that β is applied before, then α. While Jacobson's book evaluates in the opposite direction, first α, then β.

3. Cicle notation

$$\begin{array}{cccccccccc} & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \alpha = & 6 & 4 & 7 & 2 & 5 & 1 & 8 & 9 & 3 \end{array}$$

This permutation may be represented by (16)(24)(3789)(5). Notice the closed sequence of value exchanges, which we call a cicle:

$$aux \leftarrow \mathscr{L}[1];\ \mathscr{L}[1] \leftarrow \mathscr{L}[6];\ \mathscr{L}[6] \leftarrow aux$$

The value at the position 1 goes to the position 6. The previous value at position 6 must go to the position 1. The aux variable completes the cicle. Every permutation is composed of cicles, and may be represented by them, as above. There may be cicles of length one, which represent values which remain in the same position, such as 5 in the example above.

Example 1. To check if you understood cycle notation and that multiplication of permutations may be carried out in cycle form, follow in your head this example from TAOC[3]:

$$\underline{\alpha\beta = \begin{pmatrix} 0 & 2 \end{pmatrix}}\begin{pmatrix} 1 & 5 & 3 \end{pmatrix}\cdot\begin{pmatrix} 0 & 5 \end{pmatrix}\begin{pmatrix} 1 & 4 \end{pmatrix}\begin{pmatrix} 2 & 3 \end{pmatrix} = \begin{pmatrix} 0 & 3 & 4 & 1 \end{pmatrix}\begin{pmatrix} 2 & 5 \end{pmatrix}$$

[1]When a abstract algebric class is refered to, its properties are assumed with no need for further details. It means that the binary operation 'composition', represented usually with multiplication symbols, form a group over the set of permutations.

- The group itself is represented as a set-operator pair, such as: $(\mathbb{P}, \cdot)$, where $\mathbb{P}$ is the set of all permutations, and $\cdot$ is the operation symbol.
- The group is a *binary operation*, and the group inherits from its parent concept the properties:
  (1) closure: $\forall a, b \in \mathbb{P} \Rightarrow (a \cdot b) \in \mathbb{P}$
  (2) function $\mathbb{P}^2 \longrightarrow \mathbb{P}$, i.e., $\forall a, b \in \mathbb{P} \Rightarrow (a \cdot b) \in \mathbb{P}$ *and it is unique*
- A group has the following properties:

(1) Associative: $(a \cdot b) \cdot c = a \cdot (b \cdot c)\ \ \forall a, b, c \in \mathbb{P}$
(2) Neutral element: $\exists I \,|\, \forall \alpha \in \mathbb{P}, I \cdot \alpha = \alpha \cdot I = \alpha$
(3) Inverse: $\forall \alpha \in \mathbb{P}, \exists\, \alpha^{-1} \,|\, \alpha \cdot \alpha^{-1} = \alpha \cdot \alpha^{-1} = I$
(4) Some groups may be comutative (also called abelian), others are not.

[2]two famous algebra books. A good undergraduate book is Preparata.
[3]TAOC V4 7.2.1.2

4. Problems and Exercises

Problem 1. Correspondence between Notation and permutations

(1) Is the correspondence between cicle notation and tables one-to-one? How many possible table notations and cicle notations exist? Note, the question asks about the notations themselves, not the represented permutations.
   (a) How many valid strings representing table and cicle notations exist for n numbers? Tip: how many cycle representations exist which keep the same order of the numbers in the string?
(2) But, if there are always $n!$ permutations, how this is possible? Both notations represent permutations as lists of n numbers, how it is possible that the number is different from $n!$?
   (a) Are there permutations which have more than one possible cycle representation?
   (b) Are there permutations which can not be represented in cycle notation?
   (c) Are there ambigous cycle representations which represent more than one permutation?
   (d) Is there a bijection between permutations and cycle representations? Is there is not, what kind of problems arise?
   (e) How many cicle representations do exist?

Exercise 1. Prove that $(\mathbb{P}, \cdot)$ is a group by proving the 5 properties described. About the non-comutativity, show a counter example. Take care of answering *'why'*, and not *'what'*. In other words, rewritten definitions are not an answer.

Exercise 2. Given the following set of functions, order them by their complexity separating them correctly with $<$ and $=$ symbols to signify the $o()$ or $\Theta()$ operators, accepting a degree of abuse of notation for ease of writting.

$$e^n \quad n! \quad n^n \quad n \log n \quad n^e \quad \log(n!) \quad n$$

To clarify the answer format, for example, if the set of functions were $n \quad n^{0.5} \quad 2n \quad n^2$ then the answer would be:

$n^{0.5} < n = 2n < n^2$ because $n^{0.5} = o(n)$; $n = \Theta(2n)$ e $2n = o(n^2)$

Tip: you may look out for answers for some difficult steps, but show a complete proof. There many partial or incomplete proofs around, or just hints which point to a solution. You must prove formally as done in the undergrad material, using the limits (easier), or proving that there exists constants such that...

Tip: take care: proving $f = O(g)$ is not enough, because it is still possible that $f = o(g)$ or $f = \Theta(g)$, and the answer is different for each case. Also note that $f = O(g) \wedge g = O(f) \Rightarrow f = \Theta(g) \wedge g = \Theta(f)$.

Part 2. How to apply a permutation *in situ*

5. Problem definition

Notation issues are interesting and necessary to avoid traps when reading the literature or implementing solutions, but they are lateral issues. If a second vector is available to store the result, i.e., with $\Theta(n)$ aditional memory, it is easy to apply a permutation given under any notation, in time also $\Theta(n)$.

But, what if we are limited to $O(1)$ aditional memory, i.e., the permutation must be applied *in situ*, or by directly exchanging values of the input vector, without allocating a different vector for the permutated answer?

A first solution would be to generate all permutations, in time $O(n!)$. After each permutation is visited, it must be verified if it corresponds to the desired one, which can be done in time $O(n)$. There are various interesting ways to generate an enumeration of all permutations[4].

Problem 2. Enumerating and applying Permutations.

(1) Without reading TAOC, can you describe different algorithms to enumerate permutations?
(2) Describe the general idea of an algorithm to apply a given permutation in situ. Which notation appears more amenable to this task?

Even when using cycle notation, $O(1)$ aditional memory means we can not store the cycle. We can only 'walk' over the elements of the vector. Somehow, cycles must be detected, using some kind of *sentinel* concept, even if it costs some processing time.

[4]See pp 281 of TAOC vol. 4A, Combinatorial Algorithms, Part 1, Addison-Wesley, San Francisco, 2011.

6. An algorithm for permuting in situ

```
Input : list L and a permutation p, both with size n.
        p is given in table notation.
Output : the list L permuted in situ by p

for ( i = 1, i <= n, i ++ ) {      1
    k = p(i);                      n
    while ( k > i )                n+a
        k = p(k) ;                 a
    if ( k == i ) {                n
        y = L[i];                  b
        l = p(k);                  b
        while ( l != i ) {         b+c
            L[k] = L[l];           c
            k = l;                 c
            l = p(k);              c
          } //endwhile
        L[k] = y;                  b
     } //endif
} //endfor
```

Algorithm 1: In situ permutation.

The letters on the right of Algorithm 1 correspond to how many times each line runs. n is the input size.

Problem 3. As a preliminary analysis, explain in general terms:

(1) Why do we need only 3 variables, a,b,c? Couldn't each line be executed an specific number of times, thus it would be needed one variable for each line? Knuth calls the answer "Kirchoff Law".
(2) What is b?
(3) What is c? (tip: how many times each element value is reassigned?)
(4) What is a?
(5) Which one is the difficult part?

7. Complexity Analysis: Preliminaries

7.1. Canonical cycle notation.

Definition 1. The leader of a cycle is the smallest number in the cicle.

Definition 2. Canonical Notation is a *specific* cycle notation where:

(1) the cycles are ordered in decreasing order of their leader's values.
(2) the leader is the first number in the cycle.

e.g., the cycle (11,4,5,12) is represented in canonical notation as (4,5,12,11);

Example 2. An example using the definitions above:

index	1	2	3	4	5	6	7	8	9	10	11	12
table notation	8	10	9	5	4	1	2	7	3	6	11	12
usual cycles notation	$(1,8,7,2,10,6)(3,9)(4,5)(11)(12)$											
canonical cycle notation	$(12)(11)(4,5)(3,9)(1,8,7,2,10,6)$											
canonical, no parenthesis	$12,11,4,5,3,9,1,8,7,2,10,6$											

Example 3. Four permutations as examples of lexical, cycle and canonical notations. There are various cycle representations corresponding to the same permutation, and the table shows three cycle notations for each permutation. There may be more than three, as the numbers inside each cycle may be 'shifted' and the order of cycles may be change. The canonical notation is a '*canon*', a rule, to choose one specific cycle representation among them. These rules tell us to choose the rightmost representations in bold, and under these rules parenthesis may be ommited without ambiguity.

lexical	cycle	cycle	**cycle**	canonical
3214	(1,3)(2)(4)	(2)(3,1)(4)	**(4)(2)(1,3)**	4,2,1,3
4213	(4,3,1)(2)	(1,4,3)(2)	**(2)(1,4,3)**	2,1,4,3
3241	(1,3,4)(2)	(4,1,3)(2)	**(2)(1,3,4)**	2,1,3,4
3412	(1,3)(4,2)	(4,2)(1,3)	**(2,4)(1,3)**	2,4,1,3

Problem 4. Canonical vs. usual cycles notation.

(1) Can the parenthesis be ommited from the canonical notation, without ambiguity? Always?
   (a) The next cycle is *not* marked just by a decrescent pair of successive numbers, there is something else. Note, in Example 2, that 7,2 are in the same cycle, even if $7 > 2$. And in example 3, two lines show a 3-cycle with the same numbers, but in different orders. The cycles 1,4,3 and 1,3,4 represent different permutations.
   (b) Tip for (a): To determine the beggining of every cycle, with only one read pass over the vector, one needs to keep track of one variable/value. Which one?
(2) Does the canonical cycle notation corresponds one-to-one with the permutations? How many representations exist with n numbers? Are all permutations represented, uniquely? Is there a bijection between permutations and their canonical representations, one-to-one?
(3) Compare with the usual cycle representation.
(4) Are the three notations convertible? Can you describe algorithms to transform one representation to the other?

7.2. **Preliminaries.** Going back to algorthm 1:

(1) b is the number of cycles in the permutation. Considering the canonical notation, the leader of each cycle is smaller than the elements preceding it. Remember the analysis of the maximum algorithm. If for each cycle we find a new minimum, then the average count of attributions in the minimum algorithm is the same number as the average number of cycles.
(2) Each element of the vector L must be reassigned with its new value. But how many times it is assigned? Each element value is attributed only once, on line 13 or (exclusive) line 17. thus, $b + c = n$.

7.3. **Worst Case.** Let's check two extreme cases:

The right shift permutation:

index	1	2	3	...	$n-1$	n
table notation	2	3	4	...	n	1
permuted index	n	1	2	...	$n-2$	$n-1$
canonical	1	2	3	...	$n-1$	n

Its canonical representation has only one cicle with crescent values up to the end, resulting in $a = (n-1) + (n-2) + \cdots + 0 = \frac{1}{2}(n^2 - n)$, which is the worst case for a, quadratic. The same permutation has the best case for $b = 1$.

The identity permutation:

$I = [1, 2, \ldots, n]$ which in canonical notation is $[n, n-1, \ldots, 1]$, results in the worst case for b, and the best for a: $b = n$ and $a = 0$; Still linear.

Conclusion:: The worst case is first one, quadratic. The most difficult analysis is the average value of a, exactly the one involved in the worst case. We might be able to ignore it if it were irrelevant, but *it is clearly relevant.*

8. Complexity Analysis: Average Case of a

Example 4. 'a' corresponds to how many times the line 8 is executed. Lets examine an example:

index	1	2	3	4	5	6	7	8	9	10	11
lexical	11	9	7	10	4	1	8	3	6	5	2
canonical	4	10	5	3	7	8	1	11	2	9	6
cycles	|		|	|		|	|				|

The following table represents the execution of the first lines of the algorithm, from left to right and top to bottom:

i	1					2			3			4			5	6	7		8	9	10	11
k	11	2	9	6	1	9	6	1	7	8	3	10	5	4	4	1	8	3	3	6	5	2
p(k)	2	9	6	1		6	1		8	3		5	4				3					
while k>i	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
k=p(k)	*	*	*	*		*	*		*	*		*	*				*					
k==i					*						*			*								

Considering only the cicles 4,10,5 and 3,7,8, check how many times the 'while' and its attribution will be executed:

i	4			5	10
k	10	5	4	4	5
p(k)	5	4			
while (k>i)	*	*	*	*	*
k=p(k)	*	*			
k==i true			*		

i	3			7		8
k	7	8	3	8	3	3
p(k)	8	3		3		
while (k>i)	*	*	*	*	*	*
k=p(k)	*	*		*		
k==i true			*			

Problem 5. Questions about example 4:

(1) What happens when $k == i$? Why did it happen only 3 times in this example? How many times will it happen?
(2) The while is executed $n + a$ times. Which collumns represent the n term?
(3) Why does one cycle executes $k = p(i)$ two times, while the other cycle executes it 3 times? Both cycles have size 3.
(4) How much is a, in the worst case, for a cycle of size c? Which is the worst case?
(5) How to calculate the average value of a, considering all $n!$ permutations?

8.1. **The y_{ij} function.** Let's define a binary function, or index function, y_{ij}, defined in the interval $1 \leq i < j \leq n$, such that:

$$y_{ij} = \begin{cases} 1 & q(i) < q(z),\ i < z \leq j \\ 0 & otherwise \end{cases}$$

where q is the permutation represented as a vector in canonical cycle notation. In other words, y_{ij} is true if $p(i)$ is the minimum value in the subvetor $[q(i) \cdots q(j)]$. In the example:

index	1	2	3	4	5	6	7	8	9	10	11
lexical	11	9	7	10	4	1	8	3	6	5	2
canonical	4	10	5	3	7	8	1	11	2	9	6
cycles	|		|	|		|	|				|

values of y_{ij}:	count
$y_{12} = 1, y_{13} = 1,\ y_{14} = 0, y_{15} = y_{16} = \cdots = y_{1,11} = 0$	2
$y_{23} = y_{24} = y_{25} = \cdots = y_{2,11} = 0$	0
$y_{34} = y_{35} = \cdots = y_{3,11} = 0$	0
$y_{45} = 1, y_{46} = 1,\ y_{47} = \cdots = y_{4,11} = 0$	2
$y_{56} = 1,\ y_{57} = \cdots = y_{5,11} = 0$	1
$y_{67} = y_{68} = \cdots = y_{6,11} = 0$	0
$y_{78} = 1, y_{79} = 1, y_{7,10} = 1, y_{7,11} = 1,$	4
$y_{89} = y_{8,10} = y_{8,11} = 0$	0
$y_{9,10} = 1, y_{9,11} = 1$	2
$y_{10,11} = 0$	0
Total	11

First note that y_{ij} is only defined for $i < j$. Thus, if it were written as a matrix, only the upper triangular portion would be defined. Also note that $y_{ij} = 1$ implies that i and j belong to the same cycle, because, in canonical cycle notation, the next leader after the cycle will have a smaller value than all members of the cicle. E.g., given the following permutation in canonical format:

$(q(1) \ldots q(n)) = [\underbrace{5, 6, 9}, \underbrace{3, 7}, \underbrace{2}, \underbrace{1, 8, 4}]$

After the cycle lead by 5 comes 3, which is, by construction, guaranteed to be smaller than 5,6,9. Because a leader must be larger than the next leader, and smaller than all the numbers on its own cicle. After the cycle lead by 3, cames 2, and so on.

Also, notice that, by definition, $y_{ij} = 0 \Rightarrow y_{iz} = 0,\ \forall z > j$. That is, for each line of the table above, values 1 may only be in the beggining.

We need the probability that $y_{ij} = 1$, over all the n! permutations represented in canonical notation. Then, as the canonical notation has a one-to-one correspondence with the permutation set, we only need to calculate, in general, averaging over all sub-intervals of $[1, n]$, and considering all $n!$ permutations, the probability that i is the minimum value in the interval $[i, j]$. This is just 1 divided by the size of the interval, as $q(i)$ has the same probabily of being the minimum as any other element of the vector. Thus, $\bar{a} = 1/j - i + 1$, e.g., if the interval has size 3, then $1/3$ is the probability that its first element is the minimum.

8.2. **Calculating $p(y_{ij} = 1)$.** The average value of a is $\overline{a} = p(y_{ij} = 1)$

$$\overline{a} = \sum_{1 \leq i < j \leq n} \overline{y}_{ij} = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{1}{j - i + 1} = (n+1)H_n - 2n = \Theta(nlogn)$$

where $i < j$ because the variables represent the first and last index of the subinterval, respectively.

Proof. This table represents the values of $p(y_{ij} = 1)$ for each defined i, j value pair. The dots represent i, j values where y_{ij} is undefined. □

i

j

.	$1/2$	$1/3$	$\cdots$	$1/n{-}1$	$1/n$
.	.	$1/2$	$1/3$	$\iddots$	$1/n{-}1$
.	.	.	$1/2$	$1/3$	$\vdots$
.	.	.	.	$1/2$	$1/3$
.	.	.	.	.	$1/2$
.	.	.	.	.	.

Summing the whole table:

$$\begin{aligned}
\overline{a} = & \quad (n-1)\frac{1}{2} + (n-2)\frac{1}{3} + (n-3)\frac{1}{4} + \cdots + 2\frac{1}{n-1} + 1\frac{1}{n} \\
= & \quad \sum_{x=2}^{n} \frac{n-x-1}{x} = \left(\sum_{x=2}^{n} \frac{1}{x}\right)(n+1) - \left(\sum_{x=2}^{n} 1\right) \\
= & \quad (H_n - 1)(n+1) - (n-1) \\
= & \quad (n+1)H_n - 2n \\
= & \quad = \Theta(nlogn) \blacksquare
\end{aligned}$$

9. Exercises

The proof of §8 supposes permutations given in canonical notation. Was it really necessary? Couldn't the proof, including the probability calculation over the y_{ij} function table, work with the usual cycle notation, or with only one of the two canonical notation rules listed in definition 2?

Exercise 3. Explain why **each** rule of definition 2 is or isn't needed, **focusing on the y_{ij} function**.

Exercise 4. With the canonical representation, there is a bijection between the set of representations and the set of permutations. Is this fact important for the proof, and if it is, where is it used?

Exercise 5. Solve $\overline{a}$ analytically without the table diagonal counting trick. Read Concrete Mathematics pp 31-41. Specially the exercise of pp 39 and the three forms of resolution shown in pp 40-41. Justify each step referencing the relevant formulae in CM or elsewhere when applicable.

An example of "referencing":

$2 + 2 + 1 \overset{4.3}{=} 4 + 1 \overset{(a)}{=} 5$ onde the number refer to Knuth's book, chapter 4, equation 3, and (a) refer to a external formula 'glossary' which you write down separately to define formulas a,b,c,..., etc. You may also employ any different and simple notation to denote each external formulae you used.

Part 3. Algorithms to generate all permutations

10. Inversions (TAOCP Vol. 3 §5.1.1)

Let $a_1, a_2, \ldots, a_n$ be a permutation of the set $1, 2, \ldots, n$.

If $i < j$ and $a_i > a_j$, the pair (a_i, a_j) is called an *inversion* of the permutation;

for example, the permutation 3142 has three inversions: (3,1), (3,2), and (4,2). Each inversion is a pair of elements that is out of sort, so the only permutation with no inversions is the sorted permutation $1\,2\,\ldots\,n$.

The *inversion table* $b_1, b_2, \ldots, b_n$ of the permutation $a_1, a_2, \ldots, a_n$ is obtained by letting b_j be the number of elements to the left of j that are greater than j.

In other words, b_j is the number of inversions whose second component is j. It follows, for example, that the permutation:

$$5\,9\,1\,8\,2\,6\,4\,7\,3 \tag{10.1}$$

has the inversion table

$$2\,3\,6\,4\,0\,2\,2\,1\,0, \tag{10.2}$$

since 5 and 9 are to the left of 1; 5, 9, 8 are to the left of 2; etc. This permutation has 20 inversions in all. By definition the numbers b_j will always satisfy

$$0 \le b_1 \le n-1, 0 \le b_2 \le n-2, \ldots, 0 \le b_{n-1} \le 1,\ b_n = 0. \tag{10.3}$$

Perhaps the most important fact about inversions is Marshall Hall's observation that *an inversion table uniquely determines the corresponding permutation.* We can go back from any inversion table $b_1, b_2, \ldots, b_n$ satisfying (10.3) to the unique permutation that produces it, by successively determining the relative placement of the elements $n, n—1, \ldots, 1$ (in this order). For example, we can construct the permutation corresponding to (10.2) as follows: Write down the number 9; then place 8 after 9, since $b_8 = 1$. Similarly, put 7 after both 8 and 9, since $b_7 = 2$. Then 6 must follow two of the numbers already written down, because $b_6 = 2$; the partial result so far is therefore:

$$9\,8\,6\,7$$

Continue by placing 5 at the left, since $b_5 = 0$; put 4 after four of the numbers; and put 3 after six numbers (namely at the extreme right), giving

$$5986473.$$

The insertion of 2 and 1 in an analogous way yields (10.1).

11. The L Algorithm: Lexicographical ordering

Maybe the first intuitive atempt of generating all permutations is to aim towards a lexicographical ordering. At the caput of TAOC vol. 4 §7.2.1.2 "Generating all permutations", Knuth cites "We simply program $k+1 \to k$ and examine the digits of k for undesirable equalities. This gives the permutations in dictionary order too. On a second sober thought... we do need to think of something else."

Remark 1. One way to enumerate a set is in a given total order is to define a first element and a successor function, such that applying successively the successor function from the first element will generate all the set.

Example 5. Why is the solution Knuth cites inneficient? There are $n!$ permutations, why do we still worry about exponential time? Isn't incrementing k a valid successor function?

Remark 2. A vector in increasing order represents the first permutation in lexicographical ordering. E.g. 12345 is the first permutation of positive integers up to five in lexicographical ordering.

11.1. Overview of the L algorithm.

Init	Given a sorted sequence $a_1 \leq a_2 \leq \cdots \leq a_n$
L1:	[Visit] "Visit" the current permutation
L2:	[Find j] Find the rightmost consecutive pair $j, j+1$ of *increasing value* at position j.
	[Halt] if there is not a increasing value pair, $j = 0$, then halt.
L3:	[Increase a_j] From a_n to the left, find the first element $a_l > a_j$. Note that $a_{j+1} \cdots a_n$ are in decreasing order.
	Exchange $a_l \leftrightarrow a_j$
L4:	[Reverse $a_{j+1} \cdots a_n$]
	[Go L1]

Table 1 shows an execution example of the L algorithm (Algorithm 2).

11.2. Improved L Algorithm.

See the solution to exercise 1 §7.2.1.2 TAOC v. 4, and check Table 1 to verify there are simpler cases which may be streamlined to improve the algorithm.

L1: Visit the current permutation as before.

L2': [easiest hardcoded case] When L2 finds j as the last pair, there is only one element a_l to the right of a_j, thus no L3 search or L4 reversal are needed. Just perform $a_j \leftrightarrow a_l$ and go to L1.

L2.1': [easy hardcoded case] Similarly, when L2 finds j as the penultimate pair, there are two elements to the right of a_j. Instead of performing the L3 search and exchange then the L4 reversal, use 'hardcore ifs' to check for both possible possibilities and perform the exchange in L3 and the L4 reversal in a single step. Then go to L1.

L3': [easy increase] If this line is reached, there are at least 3 elements to the right of a_j. If the last element is already larger than a_j, then the L3 search is already finished. To save a few operations, the reversal of L4 will be divided in two steps: L4' reverses only a pair of values in the extremes a_{j+1} and a_n, then L4.1' reverses the 'middle'. Here, handle the reversal of the extremes and the L3 exchange in a single step. Then skip L4' and go to L4.1' which reverses only the 'middle'.

L3.1': [usual increase] Exchange a_j and a_l as L3 before.

L4': [reverse extremes] Exchange only a_{j+1} and a_n.

		Out
Init	1234	V
L2	$12\hat{3}\underline{4}$	
L3	$124\underline{3}$	
L4	1243	V
L2	$1\hat{2}\underline{4}\overset{\circ}{\underline{3}}$	
L3	$1\hat{3}\underline{42}$	
L4	1324	V
L2	$13\hat{2}\underline{4}$	
L3	$13\hat{4}\underline{2}$	
L4	1342	V
L2	$1\hat{3}\overset{\circ}{\underline{4}}\underline{2}$	
L3	$1\hat{4}\underline{32}$	
L4	1423	V

		Out
L2	$14\hat{2}\underline{3}$	
L3	$14\hat{3}\underline{2}$	
L4	1432	V
L2	$\hat{1}\underline{43}\overset{\circ}{\underline{2}}$	
L3	$\hat{2}\underline{431}$	
L4	2134	V
L2	$21\hat{3}\underline{4}$	
L3	$21\hat{4}\underline{3}$	
L4	2143	V
L2	$2\hat{1}\underline{4}\overset{\circ}{\underline{3}}$	
L3	$2\hat{3}\underline{41}$	
L4	2314	V

		Out
L2	$23\hat{1}\underline{4}$	
L3	$23\hat{4}\underline{1}$	
L4	2341	V
L2	$2\hat{3}\overset{\circ}{\underline{4}}\underline{1}$	
L3	$2\hat{4}\underline{31}$	
L4	2413	V
L2	$24\hat{1}\underline{3}$	
L3	$24\hat{3}\underline{1}$	
L4	2431	V
L2	$\hat{2}\underline{4}\overset{\circ}{\underline{3}}\underline{1}$	
L3	$\hat{3}\underline{421}$	
L4	3124	V
⋮	⋮	⋮

TABLE 1. Example of L Algorithm. The ^ points to a_j the lesser element of the rightmost pair in increasing order. The _ denote $a_{j+1} \cdots a_n$, the remaining part of the vector. The $^\circ$ denote a_l, the rightmost element larger than a_j. The V denote when the current vector value is output by the algorithm, or 'visited' in Knuth parlance.

Input: a sequence of elements in increasing order
Output: all permutations in lexicografical ordering.

L1: [Visit]

L2: [Find j]
$j \leftarrow n-1$
`while` $a_j \geq a_{j+1}$ `do` $j--$
`if` $j = 0$ `HALT`

L3: [Increment a_j]
$l \leftarrow n$
`while` $a_j \geq a_l$ `do` $l--$
$a_j \leftrightarrow a_l$

L4: [Reverse $a_{j+1} \cdots a_n$]
$k \leftarrow j+1,\ l \leftarrow n$
`do`
$a_j \leftrightarrow a_l$
$k++,\ l--$
`while` $k \geq l$
`GOTO L1:`

Algorithm 2: The L Algorithm, basic version. Generate all permutations in lexicographical order.

L4.1': [reverse the middle] Reverse the values between a_{j+1} and a_n, completing the L4 reversal.

Exercise 6. Implement the original L algorithm and the improved L algorithm (from exercise 1 section 7.2.1.2 TAOC v. 4) and count and plot the number of comparisons, attributions, and exchanges to generate all $n!$ permutations up to some value of n.

12. The P Algorithm (Gray code-like)

In the P Algorithm, the next permutation is obtained by *a single exchange of adjacent elements*, in the style of Gray Codes. These exchanges *increase or decrease the number of inversions in only one unit.*

An important remark from Knuth: some problem variations can not be solved in this way, as their valid permutation set does not have an Hamiltonian path[5] where all edges are only single exchanges of adjacent elements. For example, the six permutations of {1,1,2,2} are connected to each other as shown in figure 12.1:

$$\begin{array}{ccccccc} & & & 2112 & & & \\ 1122 & — & 1212 & \diagup\!\!\diagdown & 2121 & — & 2211 \\ & & & 1221 & & & \end{array}$$

Figure 12.1. There are no hamiltonian path between the permutations of {1,1,2,2} if only adjacent exchanges are allowed.

But such a path does exist for the set of all $n!$ possible permutations of distinct elements. Even more, the permutations may be arranged in interesting ways, such as 'truncated octaedron' of figure 12.2 for the permutations of {1,2,3,4}, with 8 hexagonal faces and 6 square faces.

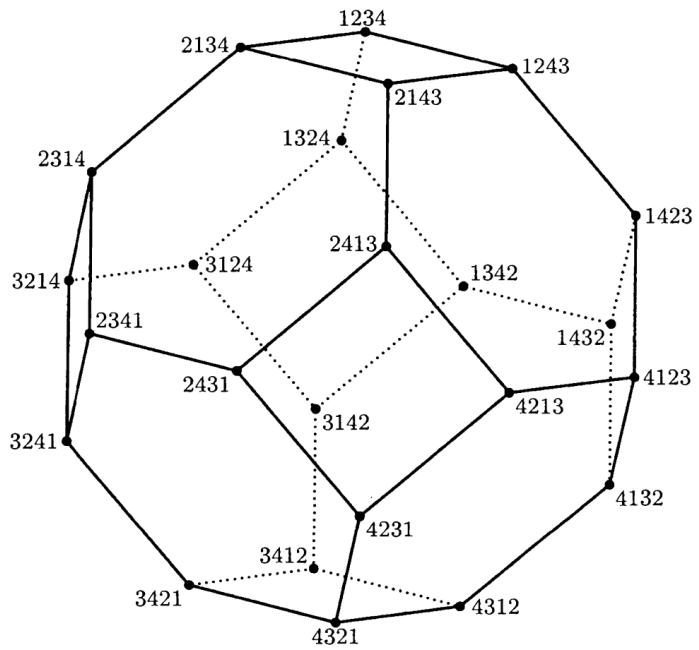

Figure 12.2. All permutations of {1,2,3,4} arranged as a polyedron. The number of inversions of a permutation is the length of a downward path from it to 1234.

[5]Note it does not need to be a Hamiltonian cycle. A Hamiltonian path suffices. `https://en.wikipedia.org/wiki/Hamiltonian_path`

The algorithm takes the sequence $1, \ldots n-1$ and inserts the number n into each permutation in all possible ways. E.g.:

1234	1324	3124	3214	2314	2134
1243	1342	3142	3241	2341	2143
1423	1432	3412	3421	2431	2413
4123	4132	4312	4321	4231	4213

Read downwards in the first collumn, upwards in the second, downwards in the third, ..., upwards in the last. Note that taking the last one, 2134, and exchanging the first two elements yield 1234, the first permutation, thus confirming the existence of a closed Hamiltonian path. As each successor permutation is obtained by an exchange of adjacent elements, the inversion table changes only one element, if read in the same order:

0000	0010	0020	0120	0110	0100
0001	0011	0021	0121	0111	0101
0002	0012	0022	0122	0112	0102
0003	0013	0023	0123	0113	0103

Also, the P algorithm keeps the inversion vector explicitly represented as a vector. Again, comparing the last and first inversion vectors, 0100 and 0000, one verifies that they differ by only one element.

12.1. **The Improved P Algorithm.** Algorithm 4 is the improved P algorithm from exercise 16, §7.2.1.2 TAOC v4. Its execution is represented by table 3. Notice that a series of outputs is streamlined at each step, and control variables and vectors are updated only when the largest value reaches the extreme positions. Also, the vector **C** does not represent exactly the inversions anymore as in the original P algorithm and Knuth's discussion.

Exercise 7. Implement the original P algorithm and the improved P algorithm and count and plot the number of comparisons, attributions, and exchanges to generate all $n!$ permutations up to some value of n. Write a table to demonstrate how the original P algorithm works and explain how it works in general terms, focusing on explaining how it keeps track of inversions. The improved P algorithm does not keep track of inversions correctly, what does it do better instead? (You may work with, exchange code, or request data from the student which is implementing the T algorithm)

13. The X algorithm: lexicographical ordering with skipping

The X algorithms still generate all permutations in lexicographical ordering. But, before visiting a permutation $a_1 \cdots a_n$, it checks $t_1(a_1), t_2(a_1, a_2), \ldots, t_n(a_1, \ldots, a_n)$, skipping all permutations with an invalid prefix. The extra loops to run the tests would only be worthy if significant branches of the permutation tree are to be skipped. On the other hand, if the number of tests is small, the overhead might be much smaller, e.g., if there are only a couple of t_is. Notice in table 4 how the vetor **A** sometimes takes invalid values temporarily, and how every prefix is tested before a permutation is visited.

Exercise 8. Implement the X (Lexicographic with restricted prefixes) algorithm and count and plot the number of comparisons, attributions, and exchanges to generate all $n!$ permutations up to some value of n, disregarding the cost of the checks t_i themselves. There will be a significant overhead compared with other algorithms. How large is the overhead? Then experiment implementing a set of simple, fast, tests which discard approximatelly 90% of all permutations. How fast does it run? Is it close to 10% of time?

Input: a sequence of elements $\mathbf{A} = a_1 \cdots a_n$ in increasing order
Output: all permutations in lexicografical ordering.

P1: [Init]
$\mathbf{C} = \mathbf{0}^n$
$\mathbf{D} = \mathbf{1}^n$

P2: [Visit]
Visit $\mathbf{A}$

P3: [Prepare Change]
$j \leftarrow n;\ s \leftarrow 0$

P4: [Ready to Change?]
$q \leftarrow c_j + d_j$
`if` $q < 0$ `go P7`
`if` $q == j$ `go P6`

P5: [change]
$a_{j-c_j+s} \leftrightarrow a_{j-q+s}$
$c_j \leftarrow q$
go P2

P6: [inc s]
`if` $j = 1$ `HALT`
`else` $s++$

P7: [switch direction]
$d_j \leftarrow -d_j$
$j--$
`go P4`

Algorithm 3: The P Algorithm. The next permutation is always obtained by a single exchange.

14. Exercises and other algorithms

Exercise 9. Implement the E (Ehrlich swaps) algorithm and count and plot the number of comparisons, attributions, and exchanges to generate all $n!$ permutations up to some value of n. Write a table to demonstrate how it works and explain how it works in general terms. Caracterize the amount of work and memory required to precompute and store the index sequence, separately from the work needed to apply the precomputed data and generate the permutations. For which range of values of n is it worth to precompute the sequence?

Exercise 10. Implement the C (cyclic shifs) algorithm and count and plot the number of comparisons, attributions, and exchanges to generate all $n!$ permutations up to some value of n. Write a table to demonstrate how it works and explain how it works in general terms. How many extra permutations does it generate? How much overhead is due to the permutations generated more than once?

P1	1234	**A**
	0000	**C**
	$++\overset{j}{+}+$	**D**
P2	$12\overset{\circ}{3}\overset{\circ}{4}$ j	Out
P3	$s\,0$	
P4	$q\,1$	
P5	$1\overset{\circ}{2}\overset{\circ}{4}3$ j	Out
	0001	**C**
P3	$s\,0$	
P4	$q\,2$	
P5	$\overset{\circ}{1}\overset{\circ}{4}23$ j	Out
	0002	**C**
P3	$s\,0$	
P4	$q\,3$	
P5	$41\overset{\circ}{2}\overset{\circ}{3}$ j	Out
	0003	**C**
P3	$s\,0$	
P4	$q\,4$	
P6	$s\,1$	
P7	$+++\overset{j}{-}$	**D**
P4	$q\,1$	
P5	$\overset{\circ}{4}\overset{\circ}{1}32$ j	Out
	0013	**C**
P3	$s\,0$	
P4	$q\,2$	

P5	$1\overset{\circ}{4}\overset{\circ}{3}2$ j	Out
	0012	**C**
P3	$s\,0$	
P4	$q\,1$	
P5	$13\overset{\circ}{4}\overset{\circ}{2}$ j	Out
	0011	**C**
P3	$s\,0$	
P4	$q\,0$	
P5	$\overset{\circ}{1}\overset{\circ}{3}24$ j	Out
	0010	**C**
P3	$s\,0$	
P4	$q-1$	
P7	$++\overset{j}{+}+$	**D**
P4	$q\,2$	
P5	$31\overset{\circ}{2}\overset{\circ}{4}$ j	Out
	0020	**C**
P3	$s\,0$	
P4	$q\,1$	
P5	$3\overset{\circ}{1}\overset{\circ}{4}2$ j	Out
	0021	**C**
P3	$s\,0$	
P4	$q\,2$	
P5	$\overset{\circ}{3}\overset{\circ}{4}12$ j	Out
	0022	**C**
P3	$s\,0$	

P4	$q\,3$	
P5	$43\overset{\circ}{1}\overset{\circ}{2}$ j	Out
	0023	**C**
P3	$s\,0$	
P4	$q\,4$	
P6	$s\,1$	
P7	$+++\overset{j}{-}$	**D**
P4	$q\,3$	
P6	$s\,2$	
P7	$++\overset{j}{-}-$	**D**
P4	$q\,1$	
P5	$\overset{\circ}{4}\overset{\circ}{3}21$ j	Out
	0123	**C**
P3	$s\,0$	
P4	$q\,2$	
P5	$\overset{\circ}{3}\overset{\circ}{4}21$ j	Out
	0122	**C**
P3	$s\,0$	
P4	$q\,1$	
P5	$32\overset{\circ}{4}\overset{\circ}{1}$ j	Out
	0121	**C**
P3	$s\,0$	
P4	$q\,0$	
P5	$\overset{\circ}{3}\overset{\circ}{2}14$ j	Out
	0120	**C**

P3	$s\,0$	
P4	$q-1$	
P7	$++\overset{j}{-}+$	**D**
P4	$q\,1$	
P5	$23\overset{\circ}{1}\overset{\circ}{4}$ j	Out
	0110	**C**
P3	$s\,0$	
P4	$q\,1$	
P5	$2\overset{\circ}{3}\overset{\circ}{4}1$ j	Out
	0111	**C**
P3	$s\,0$	
P4	$q\,2$	
P5	$\overset{\circ}{2}\overset{\circ}{4}31$ j	Out
	0112	**C**
P3	$s\,0$	
P4	$q\,3$	
P5	$42\overset{\circ}{3}\overset{\circ}{1}$ j	Out
	0113	**C**
P3	$s\,0$	
P4	$q\,4$	
P6	$s\,1$	
P7	$++\overset{j}{-}-$	**D**
P4	$q\,0$	
P5	$\overset{\circ}{4}\overset{\circ}{2}13$ j	Out
⋮	⋮	⋮

TABLE 2. Example of the P Algorithm. The $^\circ$ denote the elements to be exchanged at the next P5 step. 'Out' denote when the current vector value is output by the algorithm, or 'visited' in Knuth parlance (P2 step). **C** and **D** denote when the respective vectors are updated. $\overset{j}{\ }$ denotes, by the position of the subscript, the value of j.

Exercise 11. Implement the V algorithm (topological sorting) and count and plot the number of comparisons, attributions, and exchanges to generate all $n!$ permutations up to some value of n if the relation is empty (all permutations are accepted). Then experiment defining a relation with as simple as possible constraints to discard approximatelly 90% of all permutations. How fast does it run? Is it close to 10% of time? Write a table to demonstrate how it works and explain how it works in general terms.

Input: a sequence of elements $\mathbf{A} = a_1 \cdots a_n$ in increasing order
Output: all permutations in lexicografical ordering.

P1: [Init]
$\mathbf{C} = \{1, 2, \cdots, n\}$
$\mathbf{D} = -\mathbf{1}^n$
$z \leftarrow a_n$

P2: [Visit]
Visit $\mathbf{A}$
`if` $a_1 = z$ `go P3.5`

P3: [Hunt down]
`for` $j \leftarrow n-1 : 1$ `do`
  $a_{j+1} \leftarrow a_j$
  $a_j \leftarrow z$
  Visit $\mathbf{A}$
$j \leftarrow n-1,\ s \leftarrow 1$
`go P4:`

P3.5: [Hunt up]
`for` $j \leftarrow 1 : n-1$ `do`
  $a_j \leftarrow a_{j+1}$
  $a_{j+1} \leftarrow z$
  Visit $\mathbf{A}$
$j \leftarrow n-1,\ s \leftarrow 0$

P4: [ready to change?]
$q \leftarrow c_j + d_j$
`if` $q = 0$ `go P6`
`if` $q > j$ `go P7`

P5: [change] $a_{c_j+s} \leftrightarrow a_{q+s}$
$c_j \leftarrow q$
go P2

P6: `if` $j = 1$ `HALT`
`else` $s++$

P7: [switch direction]
$d_j \leftarrow -d_j$
$j--$
`go P4`

Algorithm 4: The P Algorithm, improved version from exercise 16. Generate all permutations.

Exercise 12. Implement the T algorithm (plain changes transitions) and count and plot the number of comparisons, attributions, and exchanges to generate the table, separated from the counts needed to apply the table and generate effectively all $n!$ permutations. Write a table to demonstrate how it works and explain how it works in general terms. For which range of values of n is it worth to precompute

P1	1234	**A**
	1234	**C**
	$----$	**D**
P2	1234	Out
P3	1243	Out
	1423	Out
	$41\overset{\circ}{2}\overset{\circ}{3}$ j	Out
	$s\,1$	
P4	$q\,2$	
P5	**4**132	Out
	1224	**C**
P3.5	1432	Out
	1342	Out
	$\overset{\circ}{1}\overset{\circ}{3}24$ j	Out
	$s\,0$	
P4	$q\,1$	
P5	3124	Out
	1214	**C**

P3	3142	Out
	3412	Out
	$43\overset{\circ}{1}\overset{\circ}{2}$ j	Out
	$s\,1$	
P4	$q\,0$	
P6	$s\,2$	
P7	$--+-$ j	**D**
P4	$q\,1$	
P5	**4**321	
	1114	**C**
P3.5	3421	Out
	3241	Out
	$\overset{\circ}{3}\overset{\circ}{2}14$ j	Out
	$s\,0$	
P4	$q\,2$	
P5	2314	
	1124	**C**
P3	2341	Out

	2431	Out
	$42\overset{\circ}{3}\overset{\circ}{1}$ j	Out
	$s\,1$	
P4	$q\,3$	
P5	**4**213	Out
	1134	**C**
P3.5	2413	Out
	2143	Out
	2134	Out
P4	$q\,4 > j$	
P7	$----$ j	**D**
P4	$q\,0$	
P6	$s\,1$	
P7	$-+--$ j	**D**
P4	$q\,0$	
P6	$j = 1$	HALT
⋮	⋮	⋮
⋮	⋮	⋮

TABLE 3. Example of the improved P Algorithm. The $^\circ$ denote the elements to be exchanged at the next P5 step. The bold denote the condition $a_1 = z$ in P2. 'Out' denote when the current vector value is output by the algorithm, or 'visited' in Knuth parlance. **C** and **D** denote when the respective vectors are updated. $_j$ denotes, by the position of the subscript, the value of j.

the sequence instead of running the P algorithm? (You may work with, exchange code, or request data from the student which is implementing the P algorithm)

15. Conventions for comparing algorithms to generate all permutations

Let's define some conventions to allow fair comparisons:

(1) Count separately:
  (a) comparisons
  (b) attributions from and to local variables which do not involve the input/output vector.
  (c) attributions which involve reading/writing from/to the input/output vector. This could correspond to 'exchanges', but let's count attibutions instead.
(2) Units: time in seconds.
  (a) Comparing time will be unfair as computers and languages may be different. It is an approximate idea.
(3) Vector sizes: from 3 to 10.
(4) Content of the vector: numbers from 1 to n.
(5) Explicit, complete output of all permutations in the specific sequence is needed.

Input: a sequence of elements $\mathbf{A} = a_1 \cdots a_n$ in increasing order; a list of tests $t_1(a_1), t_2(a_1, a_2), \ldots, t_n(a_1, \ldots, a_n)$
Output: all permutations in lexicographical ordering.

X1: [Init]
$\mathbf{L} = \{\underset{l_0}{1}, 2, \cdots, \underset{l_{n-1}}{n}, \underset{l_n}{0}\}$
$k \leftarrow 1$

X2: [enter level k]
$p \leftarrow 0$, $q \leftarrow l_0$

X3: [test $a_1, \ldots, a_k$]
$a_k \leftarrow q$
`if` $!t_k(a_1, \ldots, a_k)$ `go X5`
`if` $k = n$
Visit $\mathbf{A}$
`go X6`

X4: $u_k \leftarrow p$, $l_p \leftarrow l_q$
$k++$
go X2

X5: $[a_k++]$
$p \leftarrow q$, $q \leftarrow l_p$
`if` $q \neq 0$ `go X3`

X6: $k--$
`if` $k = 0$ `HALT`
$p \leftarrow u_k$, $q \leftarrow a_k$
$l_p \leftarrow q$
go X5

Algorithm 5: The X Algorithm. It generates all permutations in lexicographical order, skipping prefixes which fail a list of tests. The initialization of the vector $\mathbf{U}$ is irrelevant. The vector $\mathbf{L}$ has $n+1$ elements, indexed from 0 to n as indicated as subscripts.

(a) If you write an output file, it may be faster than screen output.
(b) The number of lines in the output file must be $n!$.

Sobre as apresentações:

(1) Fill the spreadsheet the professor will provide.
(2) Demonstrate how the algorithm works with an example.
(3) Our experience shows size 4 (or even 5) are needed to illustrate most algorithms. A vector of size 3 often does not require the execution of all lines.
(4) Show tables and plots for the 3 variables.

X1	1234	**A**
	12340	**L**
	$k \leftarrow 1$	
X2	k p q 1 0 1	
X3	1234	**A**
	$t_1(1)$	
X4	0XXX	**U**
	22340	**L**
X4 X2	k p q 2 0 2	
X3	1234	**A**
	$t_2(12)$	
X4	00XX	**U**
	32340	**L**
X4 X2	k p q 3 0 3	
X3	1234	**A**
	$t_3(123)$	
X4	000X	**U**
	42340	**L**

X4 X2	k p q 4 0 4	
X3	1234	**A**
	$t_4(1234)$	
	Visit **A**	1234
X6	k p q 3 0 3	
	32340	**L**
X5	k p q 3 3 4	
X3	1244	**A**
	$t_3(124)$	
X4	003X	**U**
	32300	**L**
X4 X2	k p q 4 0 3	
X3	1243	**A**
	$t_4(1243)$	
	Visit **A**	1243
⋮	⋮	⋮

TABLE 4. Example of the X Algorithm. It supposes all tests pass.